

Technical Proposal: Automated Visual & Input Verification for Volvo-CAN RPi Project

Systems Engineering Team

December 25, 2025

1 Executive Summary

This report outlines a comprehensive strategy for implementing "Playwright-style" automated testing for the 0.93-inch OLED display and GPIO-based inputs in a Raspberry Pi project. By utilizing virtual device drivers, kernel-level GPIO mocking, and environmental stress simulation, we enable a fully verifiable UI and interaction test suite capable of handling the rigors of Northern European winter operations.

2 Display Architecture: Hardware Abstraction

To achieve automated verification, we decouple the application logic from the physical I2C/SPI transport layer using a **Virtual Display Driver** pattern.

2.1 Hardware Abstraction Layer (HAL)

Utilizing the `luma.core` and `luma.emulator` libraries, we can switch between physical hardware and a software-defined "Capture" device based on the environment context.

3 Input Simulation (Buttons)

Simulating physical button presses (e.g., from a Kjell & Co kit) is critical for menu navigation.

3.1 Method B: GPIO Mockup (Kernel Integration)

For the most realistic simulation, the Linux kernel module `gpio-mockup` creates virtual GPIO pins. The test runner can "press" these buttons by writing to `debugfs`, triggering the same interrupts as physical hardware.

4 Reliability: Noise, Signal, and Timing

Automotive environments are electrically "dirty." We propose the following reliability tests:

4.1 CAN Signal Integrity

Using `can-utils` and `vcan0`, we simulate common bus failures: **Bus Flooding**, **Corrupt Payloads**, and **Dropped Frames**.

5 Environmental Simulation (The Cold Start Challenge)

Operating in Sweden at -7°C to -20°C introduces physical risks that must be addressed in the software and testing lifecycle.

5.1 Cranking Voltage Sag Simulation

During an engine start, the 12V bus often sags to 6V-8V.

- **The Test:** Simulate a "Power Sag" event where the 5V rail to the Pi drops to 4.5V for 2 seconds.
- **Requirement:** The software must detect a low-voltage interrupt and pause SD card writes to prevent filesystem corruption.

5.2 Cold-Weather Refresh Jitter

E-ink and OLED displays experience increased latency in sub-zero temperatures.

- **The Test:** Add a artificial 500ms delay in the `capture` emulator to mimic slow refresh rates.
- **Requirement:** Ensure the UI doesn't "race" (sending multiple draw commands before the first one is finished), which can lead to SPI bus lockups.

5.3 Condensation and Boot Jitter

Rapid heating of a cold car creates condensation. We recommend a "Safe Boot" delay in the `systemd` service to allow internal components to stabilize before initializing the I2C bus.

6 Hardware Mitigation Recommendations

- **Conformal Coating:** Recommend applying a thin layer of silicone-based conformal coating to the Pi and display PCB to prevent short-circuits from Swedish winter condensation.
- **Supercapacitor Buffer:** Use a small UPS HAT or supercapacitor bank to bridge the 2-second "crank gap" so the Pi doesn't reboot every time the engine starts.
- **SD Card Grade:** Use "Industrial/Endurance" rated SD cards (pSLC) which handle extreme temperature cycles better than standard consumer cards.

7 Automated Verification Workflow

The proposed "Playwright for Embedded" workflow:

1. **Stimulation:** Inject a CAN frame or trigger a virtual button.
2. **Generation:** The application renders a frame to `actual.png`.
3. **Assertion:** `pytest-image-diff` compares the output against a "Golden Master."

8 Final Recommendation

Virtualizing the framebuffer and simulating the "crank sag" and refresh latency is essential for Swedish winter reliability. This framework ensures the system remains "Automotive Grade" even when the hardware is under physical stress.